Hybrid RAG Architecture Implementation Plan

Date: 2026-10-14

Status: **Proposed implementation document. Review required before any implementation.**

This plan turns `RAG_HYBRID_ARCHITECTURE_PROPOSAL.md` into review-sized, testable implementation batches for the existing repository. It is not a replacement application and does not authorize code changes.

For this deliverable, only this document and its PDF are created. No application code, dependency, configuration, database, UI, or running service is changed. No migration, installation, functional test run, commit, or deployment is performed.

1. Executive decision

Implement an **opt-in, evidence-grounded hybrid retrieval pipeline** inside the existing FastAPI application:

```
Question + valid scope
         |
Complete scope/model/mode/corpus-aware cache identity
         |
         +--> Existing dense retrieval --+
         |                               |
         +--> Independent lexical search+
                                         |
                        Identity-based rank fusion
                                         |
                        Bounded context selection
                                         |
                  Existing relevance/prompt/provider flow
                                         |
                      Existing compatible answer output
```

First fix cache and result-identity defects. Then add independent lexical search, RRF, and backend/service parity. Make structured citations, broader production hardening, and bounded agents separately reviewable later batches.

Recommended first pilot: one trusted local process with pgvector plus PostgreSQL native full-text search. Complete FAISS lexical support and split-service capability tests before enabling those combinations. Dense-only FAISS and all other current supported configurations remain available with the feature off.

The first pilot is not a production authorization boundary. Wider access and ConPro integration require the additional security gates in Sections 11 and 13.

2. Baseline verified in the current repository

Source baseline: branch `rag`, commit `8344586`, with pre-existing working-tree changes, including the unfinished vector-result refactor. Do not discard or blanket-commit those changes during future implementation.

Area	Verified current implementation	Required evolution
API and UI	FastAPI and React/Vite/TypeScript; existing upload/ask paths.	Keep paths, answer text, and layout.
Embeddings	Local SentenceTransformers `all-MiniLM-L6-v2`, intended 384 dimensions.	Keep initially; use actual model identity in cache/index metadata.
Retrieval	Dense search, then lexical reranking of dense candidates.	Add independent lexical candidates; fusion determines hybrid order.
Storage	FAISS, pgvector, or mirrored `HybridVectorStore`.	Preserve backend selection; retrieval mode is a separate concern.
Candidate contract	`VectorSearchResult(id, score, text, metadata)` refactor.	Genuine backend IDs, provenance, and separate ranking diagnostics.
PostgreSQL search	Selects text only; wrapper manufactures positional IDs and zero scores.	Return row ID, metadata, source document, and real distance.
FAISS IDs	Derived from current list length; deletions allow collisions.	Immutable unique IDs independent of list positions.
Caches	Process-local response/retrieval caches and revision counters.	Complete keys; topology-appropriate generations and invalidation.
Ingestion	Extract, chunk, embed, clear old scope, add replacement.	Safe staging, validation, atomic publication, explicit job completion.
Split retrieval	`/search` returns only `context_chunks`; separate reranking logic.	Shared helper plus additive structured internal contract.
Evaluation	Unit tests and synthetic dashboard; previous analysis observed 76 passing tests.	New deterministic regressions and measured backend/evaluation results.

The previous test pass is historical baseline evidence, not a test run for this document or proof that hybrid retrieval works.

Verified references:

- `app\vector_store.py:29-75,111-260,263-416,419-464,546-575`
- `app\rag.py:489-513,542-595,634-698,903-1076`
- `app\retrieval_service.py:31-104`
- `app\main.py:173-239,283-354`
- `tests\test_vector_store.py:80-112`
- `tests\test_rag_pipeline.py:251-384`

Existing defaults: `TOP_K=5`, candidate pool 15 in normal configuration, `CHUNK_SIZE=800`, and `CHUNK_OVERLAP=100`. These three chunk/top-K constants are not currently environment overrides. Deployment examples may override candidate pool size, so tests must read the configured limit rather than assume 15.

No BM25 package is present in the inspected runtime requirements. PostgreSQL FTS uses the existing database; it is not BM25 and does not require a new search service.

3. Scope and invariants

Included in the initial implementation program

- Cache/filter/store identity correctness.
- Stable candidates and consistent scoped replacement.
- Backend-specific independent lexical retrieval.
- Shared bounded RRF and mode dispatch.
- Internal service compatibility and readiness.
- Regression tests, backend integration tests, evaluation, rollout, and rollback.

Explicitly excluded from the first hybrid release

- UI redesign or replacement project.
- New embedding or generative model.
- Managed search service, graph database, Redis, or agent framework.
- Automatic web/SQL tools, write actions, or autonomous ingestion.
- Unreviewed changes to grounding/provider semantics.
- Automatic exposure of the existing unauthenticated API to untrusted users.
- ConPro implementation; use `docs\CONPRO_PMIS_RAG_INTEGRATION.md` for that separately approved integration.

Required invariants

1. Feature absent/false preserves the dense retrieval/reranking route, except reviewed correctness fixes with documented changed expectations.
2. Dense and lexical retrieval use the same normalized scope and eligibility.
3. Invalid scope, unsupported topology, backend failure, and malformed service contracts never become unfiltered search or silent dense fallback.
4. Candidates are fused by identity, not by text or positional rank.
5. Same text from different sources retains separate evidence.
6. Scores are diagnostics, not confidence probabilities.
7. Candidate/context/resource bounds are tested, not merely documented.
8. Readers never combine different published corpus generations.
9. Public answer text and `question_domain` stay compatible initially.
10. Code is not written until a specific implementation batch is approved.

4. Proposed contracts and configuration

Everything newly named in this section is proposed, not implemented.

4.1 Candidate identity and ranking record

Reuse `VectorSearchResult` for backend results; extend it only where needed. Use one small typed ranking record for fusion rather than replacing every store with a plugin framework.

Proposed information	Meaning and source
`index_namespace`	Stable identity for a persistent table/corpus, or unique identity for an in-memory store instance. Never a DSN or secret hash used as a public label.
`chunk_id`	Genuine PostgreSQL row ID or immutable FAISS chunk ID.
`text`, `metadata`, `source_document`	Backend evidence and scope, preserved through ranking.
`source_version`	Actual version/ingestion identity when known; absent for legacy data rather than fabricated.
`dense_distance`	Actual backend distance; FAISS squared L2 differs from pgvector L2.
`lexical_score`	Backend-native lexical rank; not comparable across algorithms.
`dense_rank`, `lexical_rank`	Deterministic 1-based branch positions.
`fusion_score`	RRF score only; not calibrated confidence.

Fusion key: index namespace + genuine chunk ID, plus real source-version identity where available. Legacy PostgreSQL row IDs can participate without pretending they have page or revision provenance.

Keep compatibility adapters explicit. The current wrapper's invented ID/zero score fallback is unsuitable for hybrid mode. Update test doubles to return typed evidence instead of keeping an unsafe production fallback to satisfy mocks.

4.2 Proposed shared retrieval contract

The local and service paths should use the same retrieval helper:

```
Inputs:
  question, optional precomputed query vector, store
  normalized scope, retrieval mode, bounded candidate/context limits

Output:
  ordered structured candidates
  effective mode, corpus generation, ranking version
  safe branch diagnostics and timings
```

A small proposed `app\retrieval.py` is justified if it owns pure scope-aware selection/fusion used by both entry points. It must not import provider calls or create circular dependencies with `rag.py`.

Convert structured candidates to strings only at the existing prompt interface. Keep the candidate-to-CTX map separately for later citation validation.

4.3 Feature flag and capability matrix

Proposed new flag:

```
HYBRID_RETRIEVAL_ENABLED=false
```

Accept a documented strict boolean vocabulary. Reject invalid/blank values instead of treating them as false. Remember the existing `.env` loader does not override an already-set variable.

Backend/topology	Flag off	Flag on
FAISS, one process	Current dense route with fixes.	Allowed only after scoped BM25 dependency/implementation and consistency tests pass.
pgvector, one process	Dense route.	Initial pilot after FTS readiness and regressions pass.
Mirrored store, PG primary	Existing primary dense route.	Both branches use PG primary; mirror drift/update checks still required.
Mirrored store, FAISS primary	Existing primary dense route.	Requires completed FAISS lexical path; never fuse unrelated mirror IDs.
Split retrieval/multiple writers	Existing behavior is not newly certified.	Block until shared generation, protocol, authorization, and parity gates pass.

Use equal branch weights and RRF `k=60` as experiment constants initially. Retain existing candidate/top-K settings rather than adding many tuning knobs. Validate top-K and pool limits against server caps, including internal requests.

5. Work package A: correctness foundations

Approve this package first; do not bundle the entire roadmap into one change.

A1. Fix complete cache identities

Files: `app\rag.py`, existing pipeline tests, and vector-store namespace support.

Actions:

- Create a canonical sorted representation of all seven fields: tenant, collection, document, document_date, author, tag, source_system.
- Reuse field-specific normalization: `"default"` is a sentinel for the three scope fields, not automatically for every advanced metadata value.
- Include store namespace and corpus generation; two revision-1 stores must not share answers.
- Include mode, embedding/model identity, candidate/context limits, ranking and relevant prompt/provider version.

- For lexical/fused retrieval include canonical query text. A rounded vector signature alone cannot identify a lexical query.
- Preserve cache bounds/TTL and reset conventions. Cache only successful retrieval, including successful empty results; do not cache backend failures as empty evidence.

Tests: each filter changed individually; default/missing/case behavior; same question in independent stores; same vector with different lexical queries; mode/ranking changes; generation invalidation; expiry.

Exit gate: both retrieval and answer caches pass isolation regressions. The existing equivalent-embedding reuse test must be mode-aware: dense raw candidates may be reused when all relevant inputs match; hybrid lexical results must not be reused across distinct query text.

A2. Complete backend candidate fidelity and immutable IDs

Files: `app\vector_store.py`, `tests\test_vector_store.py`.

Actions:

- PostgreSQL SELECT returns real `id`, text, metadata, source_document, and `embedding <-> query` distance with deterministic ID tie-breaking.
- Preserve parameterized filter values and `_safe_identifier` validation.
- FAISS generates immutable IDs independent of current array length; reject duplicate explicit IDs and embedding/text/metadata/ID length mismatches.
- Preserve remaining IDs through clear/delete/rebuild.
- Retain legacy text deduplication at the compatible dense display boundary, but do not discard distinct-source candidates before hybrid fusion.
- Update the PG protocol/fakes to match actual supported methods rather than implying every backend already implements ID deletion or explicit IDs.

Tests: add/delete/add uniqueness; malformed shape without partial mutation; same text/different sources; actual PG distance/metadata; stable ordering.

Exit gate: no positional IDs/zero-score fabrication in hybrid candidate paths. Document intentional changes to tests that previously asserted text-only PG results.

A3. Eliminate scope-dropping compatibility retries

Files: `app\rag.py`, `app\retrieval_service.py`, tests/test doubles.

Remove broad `except TypeError` retries that call a store without filters. Validate capabilities before execution. If legacy unscoped adapters remain for trusted tests/demos, use an explicit compatibility boundary that is never selected for a scoped/hybrid request.

Tests: a backend signature error or internal TypeError cannot trigger a second, less-restricted search; all seven filters reach local and remote branches.

Exit gate: unsupported scope produces an explicit error, never broader evidence.

A4. Safe staging, replacement, and ingestion readiness

Files: `app\main.py`, `app\vector_store.py`, `app\ingestion_jobs.py`, applicable React API handlers/types, and API/store tests.

Actions:

- Replace filename-derived working-directory temporary paths with private unique files; preserve suffix only as needed for extraction and keep a safe display filename separately.
- Validate/extract/embed before publishing; assign explicit source identity.
- Define one scoped replacement operation instead of committing clear and add separately.
- Ensure queued-job file ownership/cleanup remains correct on success, failure, cancellation, and worker handoff.
- Treat a queued upload as pending, poll its existing status contract, and mark ready only on completion. No layout redesign is needed.
- Fix colliding remote document identity derivation before using remote sources in a hybrid corpus.

Tests: same-name concurrent uploads; path-like filenames; parser/embed/add failure; replacement preservation; async pending/ready/failed transitions.

Exit gate: failed replacement preserves the last published corpus; clients do not claim a queued upload is searchable.

For PostgreSQL, delete/insert/publication must share one transaction; calling the existing separately committing `clear()` and `add()` is not atomic. For FAISS, build validated next-state arrays/indexes before swapping the published state under a lock. Mirrored crash consistency is handled separately in B3.

6. Work package B: backend lexical retrieval

Depends on A1-A3 and a stable replacement contract from A4.

B1. PostgreSQL native full-text branch

Files: `app\vector_store.py`, reviewed optional migration under `scripts\`, backend integration tests, and operator documentation.

Recommended first design:

- Use `to_tsvector` on existing text with a matching GIN expression index.
- Start with explicit PostgreSQL `simple` configuration; evaluate `english` separately for prose. The indexed and queried expressions/configuration must match exactly.
- Use a parameterized safe text-to-tsquery function, such as `plainto_tsquery`, for ordinary user text. Do not interpolate raw tsquery syntax.
- Apply the same metadata/source predicates as dense search before LIMIT.
- Return genuine structured candidates ordered by native lexical rank and stable row ID.
- Use `ts_rank`/`ts_rank_cd` as native FTS ranking, not "BM25."
- Evaluate identifiers, punctuation, hyphens, dates, acronyms, accents, and non-Latin scripts; FTS tokenization does not guarantee exact-code recall.

An expression index can cover historical text without re-embedding or adding duplicate chunk rows. Any identifier-enrichment field or alternate analyzer is a measured follow-up, not an untested assumption.

Migration procedure to review, not execute now

1. Verify selected table, dimension, existing indexes, row counts, disk headroom, supported PostgreSQL/pgvector versions, and approved backup/recovery process.
2. Generate a bounded valid index name for the configured table; do not hardcode the default table or permit arbitrary identifiers.
3. Review/apply the lexical expression index separately from startup code.
4. For a large live table, evaluate `CREATE INDEX CONCURRENTLY` outside a transaction and handle interrupted/invalid indexes explicitly.
5. Verify readiness by index definition/configuration/validity, not name alone.
6. Inspect actual query plans on representative authorized filters. A planner may prefer a sequential scan for a tiny corpus; do not force an index merely to satisfy a cosmetic test.

Keep the current HNSW index and embeddings intact. Add metadata/filter indexes only when measured plans justify them.

Tests: real disposable PostgreSQL+pgvector, custom table name, legacy text rows, filtered FTS, zero matches, malformed query text, missing/wrong index readiness, and transaction recovery after errors.

B2. FAISS-associated scoped lexical branch

Files: `app\vector_store.py`, `requirements.txt` only if approved, backend tests.

Use a reviewed BM25 implementation rather than an unexplained custom scoring approximation. A package candidate such as `rank-bm25` is **not selected, installed, license-reviewed, or version-approved by this document**.

Dependency review must confirm maintenance/license, Python compatibility, reproducible versioning, tokenization behavior, scoped statistics, memory/update cost, and deterministic scores.

Actions:

- Use the same source text, metadata, and immutable IDs as FAISS.
- Compute eligibility before lexical ranking/statistics; an inaccessible document must not change scoped BM25 document frequency or average length.
- Implement a simple correctness-first scoped BM25 snapshot; benchmark cost before adding caches or an index per tenant/filter combination.
- Rebuild/stage lexical and dense state together for add/delete/clear/replace.
- Publish one immutable generation under the store's update lock.
- For sensitive authorized use, dense selection must use an eligible-vector set, not globally truncate top-K and then filter.

Tests: exact term absent from dense candidates; selective scopes; duplicate text/different IDs; empty authorized corpus; updates/failure rollback; scoped statistics unchanged by another tenant's text.

Until this passes, flag-on FAISS fails readiness clearly; dense-only remains usable. Do not label a PG-only pilot universal backend support.

B3. Mirrored backend semantics

Both branches query the configured primary store. Do not combine a FAISS list with a PG list whose row IDs differ.

Stage/validate updates, record mirror health, and define a canonical recoverable generation. There is no distributed transaction between a PostgreSQL commit and an in-memory FAISS publication. Do not promise crash-atomic mirrored writes.

For the initial PG-primary pilot, PG is the persistent authority and FAISS is rebuildable. A mirror update failure must be explicit; do not report both stores ready. Block incompatible primary switches until reconciliation completes.

Exit gate: tests demonstrate primary-only fusion, surfaced partial failures, restart reconstruction, and no cross-mirror identity fusion.

7. Work package C: fusion and shared mode dispatch

Files: proposed `app\retrieval.py` if needed, `app\rag.py`, `app\retrieval_service.py`, and focused retrieval tests.

C1. RRF algorithm contract

For each genuine candidate identity:

```
RRF = sum(1 / (60 + branch_rank))
```

Branch ranks are 1-based. Missing membership contributes zero. Deduplicate an identity within a branch before assigning ranks; retain separate provenance for identical text in different sources.

Example: dense list A,B and lexical list B,C gives:

```
A = 1/61
B = 1/62 + 1/61
C = 1/62
```

B must rank first. Preserve raw distances/lexical scores separately; do not sum them into RRF or present RRF as confidence.

Ordering: descending fusion score, existing lexical relevance signal only for fusion ties, then stable compound evidence identity. Do not re-sort hybrid results primarily by the old lexical reranker.

Per-branch bound P = configured candidate pool. With default P=15, the union is at most 30; retain at most P fused working candidates and final TOP_K=5. Enforce caps for oversized internal requests too.

C2. Mode and failure behavior

Condition	Required behavior
Flag off	Existing dense retrieval and reranking with reviewed correctness fixes.

Condition	Required behavior
Both branches successful	Fuse identities and select bounded context.
One branch successfully empty	Use the other branch; this is not a degraded error mode.
Both branches successfully empty	Existing no-relevant-information answer path.
Branch/backend failure	Sanitized logged error and approved API error response; no empty-result substitution.
Unsupported/mismatched mode	Explicit readiness/protocol rejection, not silent dense fallback.

Keep current relevance/grounding gates initially. Measure retrieval gains before the gates and false abstention after them. Do not weaken grounding to improve a headline recall metric.

Proposed error-contract review: use a sanitized retrieval-unavailable response (normally HTTP 503), distinguish invalid remote protocol (normally HTTP 502), and reject invalid request bounds through validation. Preserve existing provider fallback behavior until separately reviewed. These are proposed changes, not claims about the current API's error shape.

C3. Shared implementation boundary

Local and service entry points must call the same scope/ranking helper. Avoid copying RRF into both routes. Keep embedding responsibility explicit so remote queries are not embedded twice in a later performance cleanup.

Exit gate: deterministic rank math, ties, identity deduplication, branch errors, caps, and flag-off compatibility tests pass.

8. Work package D: internal API and topology consistency

D1. Additive `/search` contract

Files: `app\retrieval_service.py`, `app\rag.py`, typed internal models/tests.

Preserve legacy `context_chunks`. Add structured candidates and effective mode/contract/ranking/corpus-generation fields under an explicitly versioned internal protocol.

API and retrieval service agree on capabilities before flag-on traffic. An old service may still serve its compatible dense contract, but cannot claim hybrid support or silently discard scope/provenance.

Validate response types, IDs, metadata, bounds, and expected mode. Untrusted returned source identities must not enter prompts merely because HTTP was 200.

Tests: local/remote parity on one corpus; old/new peers; malformed/missing fields; filter preservation; top-K/pool caps; service timeout and explicit failure.

D2. Shared generations before multi-worker support

Local revision counters are not enough for independent writers or processes. Propose a small durable generation registry in the existing RAG database, scoped by index namespace. This is an additional schema decision requiring approval.

- Commit replacement/deletion and generation increments in the same transaction.
- Read generation and dense/lexical evidence from a consistent bounded database snapshot; retry on publication mismatch rather than cache mixed results.
- Ensure every authorized writer uses the same generation protocol.
- End read transactions promptly; recover/rollback after DB errors.
- Serialize use of a shared connection during its short transaction, or adopt an approved connection-management approach. Do not interleave transactions across worker threads on one mutable connection.

A one-process pilot may use an in-process generation only when it is the sole writer and all ingestion runs through that process. Document and verify that deployment restriction. `API_WORKERS=1` alone does not detect an externally launched multi-worker Uvicorn deployment.

Until D1/D2 pass, do not certify split/multi-worker hybrid support.

9. Work package E: provenance and structured citations

This is a separately approved follow-on, not a prerequisite claim that phase-C answers are fully fact-verified.

Files: `app\ingestion.py`, `app\db_ingestion.py`, `app\chunking.py`, `app\rag.py`, `app\schemas.py`, API/extraction tests.

Actions:

- Preserve parser-supported document/version/page/paragraph/table/row/offset locations in a typed extraction/chunk contract.
- Keep leading section-label compatibility while associating metadata with each actual chunk.
- Map prompt-local CTX IDs to retained source identities.
- Validate emitted references against those sources; never invent missing page or row identifiers.
- Add optional structured `sources` without removing answer text/domain.
- Plan extraction-version migration and selective reindexing; do not fabricate provenance for legacy rows.

Review synthesis/external-prefix grounding bypasses and `External response:` / `External knowledge:` inconsistency separately, with behavioral tests. Identity-valid citations do not prove every factual claim is supported.

Exit gate: genuine authorized source mappings and extraction fixtures for each claimed format; current clients still render the compatible text response.

10. Evaluation and test implementation plan

10.1 Test placement

Names for new files below are proposed.

Test surface	Coverage to add
`tests\test_rag_pipeline.py`	Cache scope/store/mode/query/generation, flag-off behavior, abstention/gates, provider not called on empty/irrelevant context.
`tests\test_vector_store.py`	IDs/shapes, real-result contract, scope, atomic publication, backend capabilities, mirror behavior.
`tests\test_config.py`	Strict flag parsing, enabled-backend/topology readiness, invalid values.
`tests\test_api.py`	Compatible answer contract, safe upload staging, explicit failures, async completion, scope propagation.
Proposed `tests\test_retrieval.py`	RRF arithmetic, rank deduplication, stable ties, pool caps, empty/failing branches.
Proposed `tests\test_retrieval_service.py`	Internal protocol, local/remote parity, invalid/mismatched service payloads, bounds.
Proposed `tests\test_pgvector_integration.py`	Real PostgreSQL FTS/HNSW/filters/transactions/index readiness/generations on disposable tables.
Existing ingestion/chunking tests	Later provenance/format fixtures and backward-compatible labels.

Follow existing monkeypatch conventions: patch the importing module, reset `rag._clear_caches()`, reset API singletons/metrics/feedback, stub heavy models and providers in unit tests. Unit tests must not download a model or call a provider.

Real database tests require an explicitly designated disposable test database, unique table namespace, cleanup, and permissions that cannot touch production. Report skipped backend tests plainly; unit mocks cannot substitute for them.

10.2 Required evaluation artifact

Create a small versioned, non-sensitive corpus and a labeled question set with:

- Query ID/text, authorized scope, expected relevant source/version/chunk IDs.
- Exact-reference, semantic, numerical, multilingual, conflicting, and unanswerable categories.
- Source/extraction/model/ranking/config versions.
- Frozen dense baseline and reproducible corpus hash.

Use dense-only, lexical-only, and dense+lexical+RRF ablations. A later learned reranker is a separate experiment, not part of the initial baseline.

Measure Recall@5, MRR/nDCG where labels support them, exact-reference/semantic subsets, selected context, abstention, citation validity, latency, index cost, and memory. Measure final answers separately from candidate recall.

Do not replace the synthetic dashboard with invented measured values. Wire versioned evaluation output into it only in a separately tested reporting step.

10.3 Proposed pass/fail gates

Gate	Required outcome
Compatibility	Existing expected valid dense behavior passes; changed bug expectations are documented.
Isolation	Zero cross-filter/cross-store/unauthorized candidates, contexts, cache hits, or citations in the regression matrix.
Fidelity	Genuine stable identities/metadata/distances and recoverable scoped replacement.
Retrieval	Macro Recall@5 at least frozen dense baseline; initial target +5 percentage points on exact-reference subset.
Semantic protection	Reject material semantic-subset regression; agree numeric tolerance on the labeled corpus before enabling.
Latency	Initial warm retrieval p95 target no more than 1.25x dense baseline under identical workload/hardware.
Failures	Branch errors are explicit; no unfiltered retry or readiness bypass.
Topology	Every enabled backend/topology passes its own real integration/parity/generation tests.
Rollback	Flag-off works without deleting corpus data or re-embedding.

Targets are proposed, not measured guarantees. Keep the feature off if quality does not improve or constraints fail.

10.4 Future validation commands

These commands are documented for the approved implementation stage, not run now. Run the smallest changed-file selectors first and escalate to the full suite at release gates.

```
.\.venv\Scripts\python.exe -B -m pytest tests\test_vector_store.py tests\test_rag_pipeline.py
        tests\test_config.py -q
.\.venv\Scripts\python.exe -B -m pytest tests\test_api.py -q
.\.venv\Scripts\python.exe -B -m pytest -q
```

When the proposed retrieval/service test files exist, include them in the same targeted runner. Run real-PG tests only against the disposable test environment.

For actual frontend changes:

```
Set-Location frontend
npm run build
```

Use existing lint/typing tools when available; do not install tools merely to export this document. Do not run automatic whole-repository formatting over unrelated dirty files.

11. Security and production release gates

Hybrid search does not add authentication or make client-supplied tenant IDs authoritative. Before untrusted access or ConPro data:

- Implement authenticated server-derived scope and source eligibility.
- Apply scope to both branches before candidate/context exposure.
- Scope jobs/feedback/metrics appropriately.
- Restrict connectors and database credentials; do not accept arbitrary URLs, SQL, or source locators as an authorization shortcut.
- Bound downloads, extraction, query sizes, queues, candidate counts, and prompts.
- Add build-context secret exclusions and sanitize logs/errors.
- Revalidate authorization and source generations on cache hits and before provider dispatch/answer return where required.
- Approve external processing; `auto` mode can disclose prompts to both providers.

These are separate reviewable tasks, not implemented protections. ConPro's policy/version/source contract is defined in the integration document and must not be replaced with a raw `/ask` proxy.

12. File-level change inventory

A=Correctness Foundations
B=Backend Lexical Retrieval
C=Fusion+SHared Module Dispatch
D=Internal API+Topology Consistency
E=Work Package

All entries describe future changes.

File/surface	Proposed change	Batch
`app\config.py`	Strict flag, capability validation, existing defaults preserved.	C/D
`app\vector_store.py`	Namespace/IDs/results; scoped replacement; lexical branches; primary semantics.	A/B/D
`app\rag.py`	Complete cache keys, no scope-dropping retry, shared mode dispatch, evidence retention.	A/C/D/E
Proposed `app\retrieval.py`	Small shared typed selection/fusion boundary if needed.	C
`app\retrieval_service.py`	Shared filters/ranking, additive validated protocol and caps.	A/C/D

File/surface	Proposed change	Batch
`app\main.py`	Unique staging, source identity, scoped replacement/readiness integration.	A/D
`app\ingestion_jobs.py`	Correct temporary-file ownership and job failure/completion behavior; later durability.	A/later
`app\schemas.py`	Later additive sources/status after response-contract approval.	E
Extraction/chunking modules	Later real source locations and extraction versioning.	E
`app\streamlit_demo.py`	Ensure shared scoped retrieval/ingestion; preserve demo role.	C/E
`frontend\RAGQAApp.tsx`, typed API files	Only necessary async/API correctness; no layout redesign.	A/E
`scripts\`	Explicit reviewed FTS migration; optional shared-generation migration.	B/D
`requirements.txt`	BM25 dependency only after review, if selected.	B2
`tests\`	Focused regressions, backend/service tests, fixtures.	Every batch
`README.md`, environment/deployment examples	Defaults, supported capabilities, migration/readiness, evaluation, rollback.	Every release
CI definition	Run existing targeted/full checks and explicit real-backend job when available.	Release gate

Think of the roadmap like this:

Batch Purpose
A Correctness foundations
B Lexical retrieval (PG + FAISS)
C Fusion + hybrid mode
D Internal API + multi-worker consistency
E Provenance + structured citations

Streamlit is Not needed

Do not edit stale nested checkouts or historical frontend copies. Integrate the existing dirty vector-store refactor rather than overwriting it.

13. Sequencing, approval gates, and deliverables

Review-sized batch	Prerequisites	Deliverable and stop point
A1-A3	Explicit implementation approval.	Cache/result/ID/scope regressions and fixes; dense behavior review. Stop for review.
A4	Stable identity/replace interface.	Safe ingestion and ready-state correctness. Stop for review.
B1 + C	Foundations, approved PG migration and benchmark corpus.	PG one-process hybrid experiment, default off. Do not enable beyond approved pilot.

Review-sized batch	Prerequisites	Deliverable and stop point
B2/B3	Dependency/recovery decisions and foundations.	FAISS/mirrored capability tests and documentation. Enable only passing combinations.
D	Shared-generation/protocol schema approval.	Split/multi-worker parity and consistency. Stop until real integration tests pass.
E	Evidence/extraction response-contract approval.	Genuine structured sources and reindex plan. Review grounding separately.
Production hardening	Authentication, quotas, durability, telemetry, approved providers.	Authorized monitored rollout; ConPro remains separately gated.
Optional agentic work	Measured unresolved need after evaluation.	Separate proposal; no agent automatically included.

Engineering effort cannot be credibly estimated in days before dependency, deployment, corpus, and migration decisions are settled. Keep each batch small enough for independent review and rollback rather than inventing a schedule.

14. Rollout and rollback runbook

Before enablement

1. Approve a specific batch, backend/topology, test corpus, and changed behavior.
2. Capture baseline questions/results and current configuration without secrets.
3. Apply only approved additive schema/index work; verify readiness and recovery.
4. Pass tests with the flag off, then on for the selected backend/topology.
5. Benchmark on a non-sensitive copy; compare quality/latency and failure paths.
6. Enable only in the approved pilot; observe branch latency, hits, abstention, failures, freshness, and mirror health with sanitized telemetry.

Do not restart currently running services or change live flags during document review.

Rollback after a future deployment

- Set the proposed flag false and restart affected processes through the approved deployment procedure; configuration is read at import.
- Ensure API/retrieval peers agree on dense mode; clear or safely namespace caches by mode/ranking/generation.
- Preserve chunk rows, vectors, source metadata, and generation records.
- Leave optional FTS indexes inert initially. Remove only reviewed optional indexes later if required; never drop a corpus table to roll back search.
- Correctness/authentication fixes remain in place; rollback must not restore unfiltered retries, cache leakage, or unsafe upload paths.
- Drain/reconcile pending publication safely and verify dense results on the baseline questions.

Additive data migrations and application rollback compatibility must be tested together; reverting a code commit is not a database recovery strategy.

15. Review checklist and recommended first approval

Review decisions:

- [ ] Accept default-off hybrid retrieval and independent backend capability gates.
- [ ] Approve A1-A3 as the first implementation batch, or specify a narrower batch.
- [ ] Confirm single-process PG as the first experimental deployment.
- [ ] Approve the labeled non-sensitive corpus and quality/latency targets.
- [ ] Choose/review the FAISS BM25 dependency before B2.
- [ ] Approve FTS configuration/index migration before B1 deployment.
- [ ] Approve shared-generation schema before split/multi-worker enablement.
- [ ] Keep citation/API/extraction and production security changes separately reviewable.
- [ ] Keep ConPro integration, provider decisions, and any UI hook separately approved.

Recommended next authorization: A1-A3 only, followed by a test/results review. That first authorization would not enable hybrid search, install a BM25 package, apply a migration, restart a live service, or authorize the remaining roadmap.

Reviewing this document does not itself authorize implementation. Wait for an explicit instruction naming the approved batch.

Deliverables:

- `docs\RAG_HYBRID_IMPLEMENTATION_PLAN.md`
- `docs\RAG_HYBRID_IMPLEMENTATION_PLAN.pdf`

Identical review copies are also saved in the ConPro repository's `docs` folder, following the previous document delivery convention. This does not implement or modify ConPro.

Design references in the repository root: `RAG_HYBRID_ARCHITECTURE_PROPOSAL.md` and `RAG_CODEBASE_ANALYSIS.md`. The ConPro-specific design remains `docs\CONPRO_PMIS_RAG_INTEGRATION.md`.